Taller PL/SQL

Nombres: Erik Yulliam Bautista Arévalo, Felipe López Almanza, Mariana Camila Durán Piñeros

Parte 1

1.1

```
Procedure SP_REGISTRAR_PRODUCTO compilado

-> id devuelto: 1016
ERROR: el precio de venta (9000) debe ser mayor al costo unitario (10000)
  -> id devuelto: NULL (rechazado)
ERROR: la categoria 99 no existe
  -> id devuelto: NULL (rechazado)
ERROR: ya existe un producto con el codigo HM-0001
  -> id devuelto: NULL (rechazado)

Procedimiento PL/SQL terminado correctamente.
```

Solución

SP_REGISTRAR_PRODUCTO valida primero que el precio supere el costo y que la categoría exista (con COUNT(*), que siempre devuelve una fila), y solo después toma el id de SEQ_PRODUCTO, para que un dato inválido no gaste un número de la secuencia. Cada regla es una excepción propia que se levanta con RAISE y se atrapa en el mismo procedimiento, que imprime el motivo y deja el id de salida en NULL. El código duplicado se controla con la restricción UK_PRODUCTO_CODIGO (atrapando DUP_VAL_ON_INDEX) y no con un SELECT previo, porque otra sesión podría insertar el mismo código entre la consulta y el INSERT. En la captura el primer registro recibe el id 1016 y los otros tres casos (precio menor al costo, categoría 99 y código HM-0001 repetido) se rechazan con su mensaje.

1.2

```
Procedure SP_AJUSTAR_STOCK compilado

>> ENTRADA 10 | prod 1001 | bod 1
   saldo final: 130
>> ENTRADA 5 | prod 1003 | bod 2
   saldo final: 5
>> SALIDA  30 | prod 1001 | bod 1
   saldo final: 100
>> AJUSTE  95 | prod 1001 | bod 1
   saldo final: 95
>> SALIDA  1 | prod 1011 | bod 4
ERROR: no existe el producto 1011 en la bodega 4: no se puede registrar una SALIDA
   saldo final: NULL (rechazado)
>> SALIDA  10 | prod 1005 | bod 1
ERROR: stock insuficiente en bodega 1 para el producto 1005: disponible 4, solicitado 10
   saldo final: NULL (rechazado)
>> ENTRADA 1 | prod 1001 | bod 5
ERROR: la bodega 5 esta inactiva y no puede recibir entradas
   saldo final: NULL (rechazado)
>> TRASLADO1 | prod 1001 | bod 1
ERROR: tipo de movimiento invalido (TRASLADO). Use ENTRADA, SALIDA o AJUSTE
   saldo final: NULL (rechazado)
--- movimientos generados ---
21 | 1001 | bod 1 | ENTRADA  | 10 | Prueba taller
22 | 1003 | bod 2 | ENTRADA  | 5 | Prueba taller
23 | 1001 | bod 1 | SALIDA   | 30 | Prueba taller
24 | 1001 | bod 1 | AJUSTE   | 95 | Conteo fisico: existencia 100 -> 95. Prueba taller

Procedimiento PL/SQL terminado correctamente.
```

Solución

SP_AJUSTAR_STOCK valida tipo, cantidad, producto y bodega, y lee la fila de EXISTENCIA con SELECT ... FOR UPDATE: sin ese bloqueo, dos sesiones podrían leer el mismo saldo y restar ambas, perdiendo una de las actualizaciones. La regla de no dejar el stock en negativo se comprueba sobre ese valor ya bloqueado. Una ENTRADA o un AJUSTE crean la combinación de bodega y producto si no existe, y una SALIDA sin combinación se rechaza. Interpretamos el AJUSTE como conteo físico (la existencia queda igual a la cantidad indicada), porque MOVIMIENTO_INVENTARIO no admite cantidades negativas. La captura parte de 120 unidades en el producto 1001 y muestra los saldos 130, 5, 100 y 95, más los cuatro rechazos con su mensaje y los movimientos 21 a 24 registrados.

1.3

```
Procedure SP_CREAR_PEDIDO compilado

>> cliente 501
   pedido devuelto: 8
>> cliente 506
ERROR (CLIENTE INACTIVO): el cliente 506 no puede crear pedidos
   pedido devuelto: NULL (rechazado)
>> cliente 505
ERROR (SIN CUPO): el cliente 505 tiene el cupo de credito agotado
   pedido devuelto: NULL (rechazado)
>> cliente 999
ERROR: el cliente 999 no existe
   pedido devuelto: NULL (rechazado)

Procedimiento PL/SQL terminado correctamente.
```

Solución

SP_CREAR_PEDIDO consulta en un solo SELECT si el cliente está activo y cuánto cupo tiene (cupo menos saldo), y valida en este orden: existencia, estado activo y cupo mayor que cero. Cada rechazo es una excepción distinta (e_cliente_inactivo y e_sin_cupo) con su propio mensaje, como pide el enunciado. El cupo se calcula dentro del procedimiento y no con FN_CUPO_DISPONIBLE porque esa función se crea después, en la Parte 2, y referenciarla aquí dejaría el procedimiento inválido. La captura muestra que el cliente 501 recibe el pedido 8, mientras que el 506 (inactivo), el 505 (cupo agotado) y el 999 (inexistente) se rechazan con el id en NULL.

Parte 2

2.1

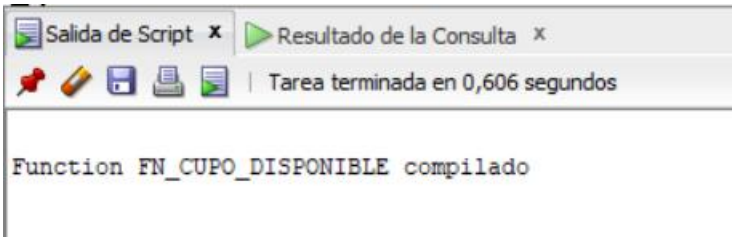
```
Function FN_STOCK_DISPONIBLE compilado
```

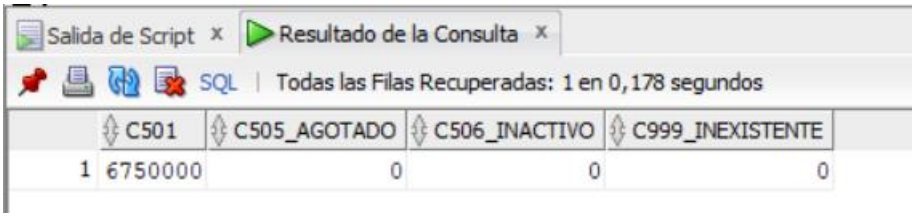


Solución

FN_STOCK_DISPONIBLE usa SUM con NVL en lugar de SELECT INTO, porque un agregado sin GROUP BY siempre devuelve una fila: así no hay NO_DATA_FOUND y el NVL convierte el NULL en 0, que es lo que el enunciado pide para un producto inexistente. Si la bodega llega nula suma todas, y si llega con valor suma solo esa; se separaron en dos consultas con IF para que cada una tenga un plan claro. La captura muestra 160 para el consolidado del producto 1001, 120 para la bodega 1 y 0 para la bodega 4, el producto 1015 (sin existencias) y el 9999 (inexistente), sin ninguna excepción.

2.2

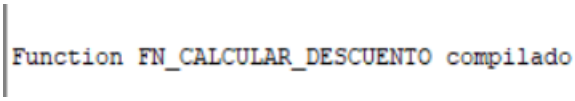


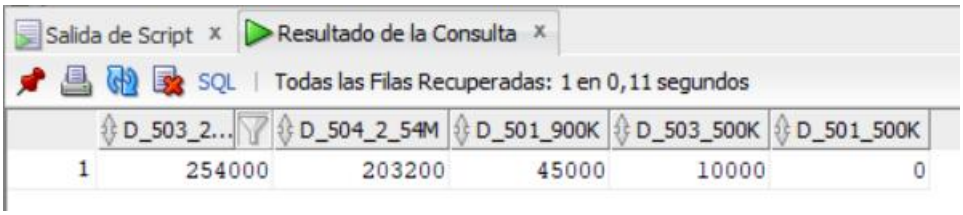


Solución

FN_CUPO_DISPONIBLE calcula cupo menos saldo con GREATEST(..., 0) para que nunca sea negativo, y resuelve la regla del cliente inactivo con un CASE dentro de la misma consulta, de modo que la tabla se lee una sola vez. Si el cliente no existe devuelve 0, igual que en 2.1, para que una consulta masiva no se aborte por una sola fila. La captura muestra 6.750.000 para el cliente 501 (8.000.000 de cupo menos 1.250.000 de saldo) y 0 para el 505 (cupo agotado), el 506 (inactivo) y el 999 (inexistente).

2.3





	D_503_2...	D_504_2_54M	D_501_900K	D_503_500K	D_501_500K
1	254000	203200	45000	10000	0

Solución

FN_CALCULAR_DESCUENTO calcula primero el porcentaje acumulado y al final lo convierte en pesos, porque el enunciado pide el valor del descuento y no el porcentaje. Los tramos se evalúan con >= 2.000.000 y >= 800.000 en ese orden, no con BETWEEN, para que un monto con decimales como 1.999.999,50 no quede fuera de ninguno. Entendimos "acumulables" de forma literal: el 2% por no tener saldo se suma incluso cuando el tramo es 0%, y el tope del 10% se aplica con LEAST (hoy no recorta nada, pues el máximo posible es 8 más 2). En la captura, el cliente 503 (sin saldo) con 2.540.000 obtiene 254.000, el 504 (con saldo) obtiene 203.200, y los montos de 900.000 y 500.000 dan 45.000, 10.000 y 0.

2.4



	ID_CLIENTE	NOMBRE	CUPO_DISPONIBLE
1	504	Obras Civiles del Valle	8600000
2	501	Constructora Andes SAS	6750000
3	507	Distribuidora Caribe	5200000
4	503	Ferreteria La Esquina	2000000
5	502	Electricos del Norte Ltda	300000
6	505	Mantenimientos Tecnicos	0

Solución

La consulta lista los clientes activos con FN_CUPO_DISPONIBLE y los ordena de mayor a menor; en la captura el orden es 504, 501, 507, 503, 502 y 505, y el 506 no aparece por estar inactivo. Una función puede invocarse desde SQL porque recibe parámetros y devuelve un solo valor de tipo SQL, así que el motor la trata como una expresión escalar y la ejecuta en PL/SQL por cada fila. La restricción es que no puede modificar la base de datos desde una consulta (ni INSERT, UPDATE, DELETE o DDL, ni COMMIT o ROLLBACK, errores ORA-14551 y ORA-14552); FN_CUPO_DISPONIBLE cumple porque solo hace SELECT.

Parte 3

3.1

```
DECLARE
    CURSOR cur_bajo_stock IS
        SELECT codigo, producto, stock_total, stock_minimo, stock_minimo - stock_total AS faltantes
        FROM   V_STOCK_CONSOLIDADO
        WHERE  stock_total < stock_minimo
        ORDER BY codigo;

    vrProducto  cur_bajo_stock%ROWTYPE;
    vnContador  NUMBER := 0;
BEGIN
    DBMS_OUTPUT.PUT_LINE(' ------ productos por debajo del stock minimo ------');
    DBMS_OUTPUT.PUT_LINE(RPAD('CODIGO', 10) || RPAD('NOMBRE', 34) || LPAD('STOCK', 8) || LPAD('MINIMO', 8) || LPAD('FALTAN', 8));

    OPEN cur_bajo_stock;
    LOOP
        FETCH cur_bajo_stock INTO vrProducto;
        EXIT WHEN cur_bajo_stock%NOTFOUND;

        vnContador := vnContador + 1;
        DBMS_OUTPUT.PUT_LINE(RPAD(vrProducto.codigo, 10) || RPAD(SUBSTR(vrProducto.producto, 1, 32), 34) || LPAD(vrProducto.stock_total, 8) ||
                             LPAD(vrProducto.stock_minimo, 8) || LPAD(vrProducto.faltantes, 8));
```

Salida de Script

Tarea terminada en 0.286 segundos

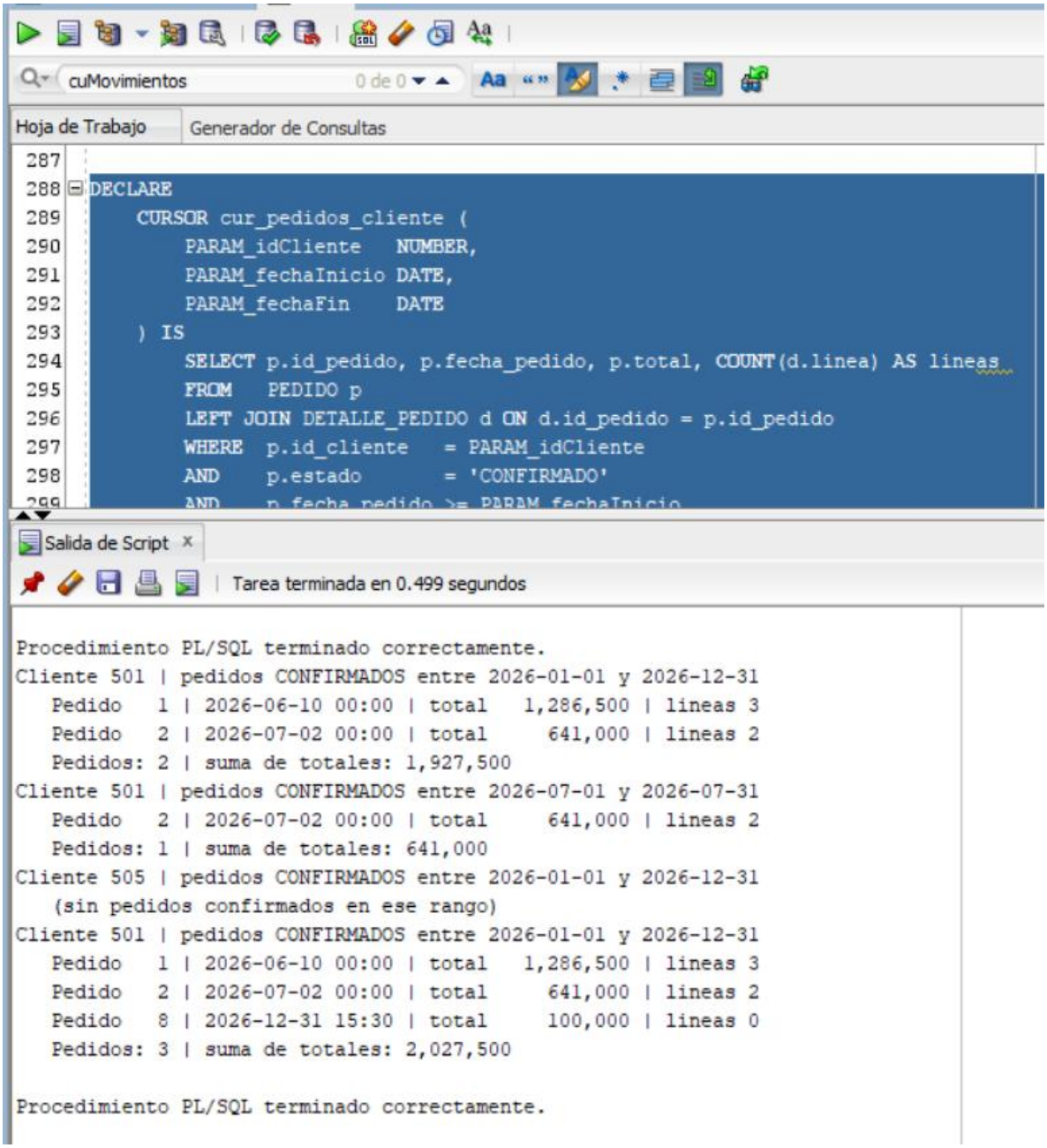
```
------ productos por debajo del stock minimo ------
CODIGO    NOMBRE                              STOCK  MINIMO  FALTAN
FG-0002   Cinta metrica 8 metros                  9      25      16
HM-0004   Alicate universal 8 pulgadas            0      20      20
ME-0001   Cable THHN 12 AWG metro               470     500      30
-----------------------------------------------------------------
Total de productos por debajo del minimo: 3

Procedimiento PL/SQL terminado correctamente.
```

Solución

En este ejercicio usamos un cursor explícito sobre la vista V_STOCK_CONSOLIDADO, ya que esta vista suma el stock de todas las bodegas por producto. El cursor trae solo los productos cuyo stock total es menor que el stock mínimo, y calcula las unidades faltantes (mínimo menos stock). El bloque abre el cursor con OPEN, lee un producto por vuelta con FETCH, imprime código, nombre, stock, mínimo y faltantes, y se detiene cuando el atributo del cursor, %NOTFOUND indica que ya no hay más filas. Al final cierra el cursor con CLOSE e imprime el total. En la captura salen 3 productos: FG-0002 (faltan 16), HM-0004 (faltan 20) y ME-0001 (faltan 30).

3.2



```
DECLARE
    CURSOR cur_pedidos_cliente (
        PARAM_idCliente   NUMBER,
        PARAM_fechaInicio DATE,
        PARAM_fechaFin    DATE
    ) IS
        SELECT p.id_pedido, p.fecha_pedido, p.total, COUNT(d.linea) AS lineas
        FROM   PEDIDO p
        LEFT JOIN DETALLE_PEDIDO d ON d.id_pedido = p.id_pedido
        WHERE  p.id_cliente   = PARAM_idCliente
        AND    p.estado       = 'CONFIRMADO'
        AND    p.fecha_pedido >= PARAM_fechaInicio
```

```
Procedimiento PL/SQL terminado correctamente.
Cliente 501 | pedidos CONFIRMADOS entre 2026-01-01 y 2026-12-31
   Pedido   1 | 2026-06-10 00:00 | total   1,286,500 | lineas 3
   Pedido   2 | 2026-07-02 00:00 | total     641,000 | lineas 2
   Pedidos: 2 | suma de totales: 1,927,500
Cliente 501 | pedidos CONFIRMADOS entre 2026-07-01 y 2026-07-31
   Pedido   2 | 2026-07-02 00:00 | total     641,000 | lineas 2
   Pedidos: 1 | suma de totales: 641,000
Cliente 505 | pedidos CONFIRMADOS entre 2026-01-01 y 2026-12-31
   (sin pedidos confirmados en ese rango)
Cliente 501 | pedidos CONFIRMADOS entre 2026-01-01 y 2026-12-31
   Pedido   1 | 2026-06-10 00:00 | total   1,286,500 | lineas 3
   Pedido   2 | 2026-07-02 00:00 | total     641,000 | lineas 2
   Pedido   8 | 2026-12-31 15:30 | total     100,000 | lineas 0
   Pedidos: 3 | suma de totales: 2,027,500

Procedimiento PL/SQL terminado correctamente.
```

Solución

En este ejercicio declaramos un cursor con tres parámetros: el cliente, la fecha inicial y la fecha final. Trae los pedidos CONFIRMADOS de ese cliente dentro del rango, con su total y la cantidad de líneas (cuenta las filas de DETALLE_PEDIDO de cada pedido). El bloque recorre el cursor con un ciclo FOR, muestra cada pedido y al final cuántos fueron y la suma de sus totales. Lo probamos con el cliente 501 en 2026 (salen los pedidos 1 y 2), con otro rango (solo julio), con un cliente sin pedidos y con un pedido temporal del 31 de diciembre a las 15:30, que sí entra al rango. Ese pedido temporal se deshace con ROLLBACK.

3.3

```
371 DECLARE
372     CURSOR cur_productos_cat (PARAM_idCategoria NUMBER) IS
373         SELECT p.codigo, p.precio_unitario, p.costo_unitario, c.margen_minimo
374         FROM   PRODUCTO p
375         JOIN CATEGORIA c ON c.id_categoria = p.id_categoria
376         WHERE  p.id_categoria = PARAM_idCategoria
377         AND    p.activo = 'S'
378         ORDER BY p.codigo
379         FOR UPDATE OF p.precio_unitario;
380
381     vrProducto      cur_productos_cat%ROWTYPE;
382     vnIdCategoria         NUMBER := 1;      -- categoria a ajustar
383     vnPorcentajeAjuste    NUMBER := -22;
384     vnPrecioNuevo         NUMBER;
385     vnPrecioMinimo        NUMBER;
386     vnActualizados        NUMBER := 0;
```

Salida de Script

Tarea terminada en 0.298 segundos

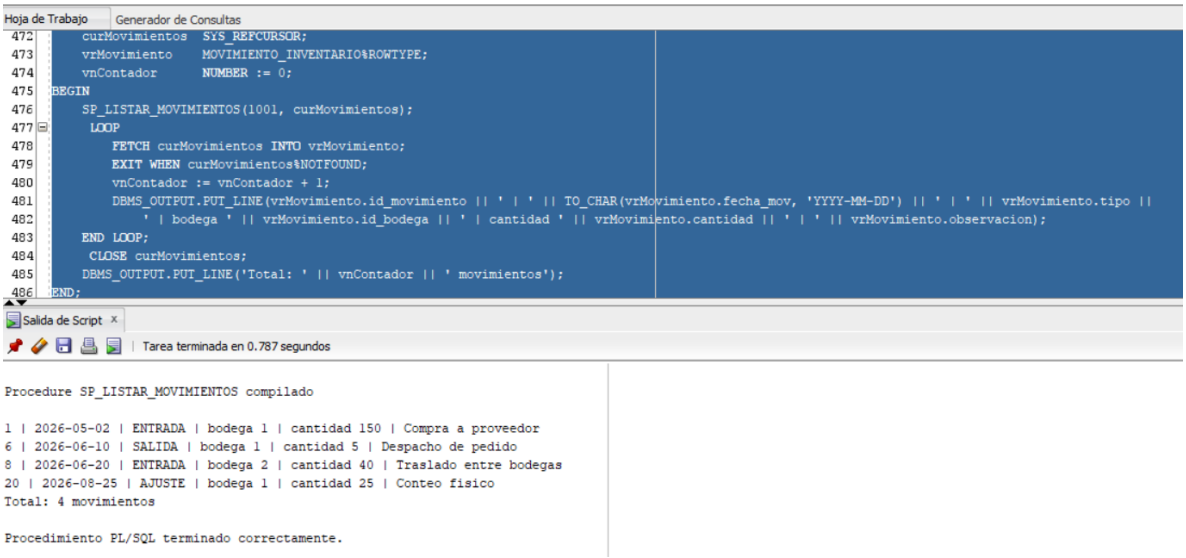
```
Ajuste de precios | categoria 1 | porcentaje -22%
CODIGO       ANTERIOR      NUEVO     MINIMO  RESULTADO
HM-0001     45,000.00  35,100.00  35,000.00  ACTUALIZADO
HM-0002     62,000.00  48,360.00  47,500.00  ACTUALIZADO
HM-0003     38,000.00  29,640.00  30,000.00  OMITIDO (quedaria bajo el minimo)
Actualizados: 2 | Omitidos: 1
Precios despues del ajuste:
   HM-0001     35,100.00  activo=S
   HM-0002     48,360.00  activo=S
   HM-0003     38,000.00  activo=S
   HM-0004     33,000.00  activo=N

Procedimiento PL/SQL terminado correctamente.
```

Solución

En el 3.3 usamos un cursor FOR UPDATE que recorre los productos activos de una categoría y trae también el margen mínimo de esa categoría. Para cada producto calculamos el precio nuevo (precio por 1 más el porcentaje) y el precio mínimo permitido (costo por 1 más margen/100). Si el precio nuevo no baja del mínimo, lo actualizamos con WHERE CURRENT OF, que modifica justo la fila que acabamos de leer. Si baja del mínimo, no lo tocamos y lo informamos. Al abrir el cursor, FOR UPDATE bloquea las filas, y el bloqueo dura hasta el COMMIT o ROLLBACK. Con la categoría 1 y -22%, HM-0001 (45.000 a 35.100) y HM-0002 (62.000 a 48.360) se actualizan. HM-0003 se omite porque quedaría en 29.640, bajo el mínimo de 30.000. El inactivo HM-0004 no se toca. Al final mostramos los precios y hacemos ROLLBACK.

3.4



Solución

En el 3.4, el procedimiento SP_LISTAR_MOVIMIENTOS recibe un producto y devuelve, por un parámetro de salida de tipo SYS_REFCURSOR, una consulta abierta con sus movimientos ordenados por fecha y por id. SYS_REFCURSOR es un tipo de dato que trae Oracle para guardar una referencia a una consulta. La variable no contiene las filas, sino que apunta a una consulta ya abierta, y quien la recibe va leyendo las filas. Permite que el procedimiento entregue los datos sin cargarlos todos en memoria.

El procedimiento solo abre la consulta con OPEN ... FOR, no lee ni cierra nada. Para probarlo, un bloque anónimo llama al procedimiento con el producto 1001, recibe el cursor y lo lee fila por fila con FETCH. Imprime cada movimiento, y al final lo cierra con CLOSE y muestra el total. En la captura salen 4 movimientos (ids 1, 6, 8 y 20).

3.5

```
506 DECLARE
507     TYPE tbIdProducto  IS TABLE OF PRODUCTO.id_producto%TYPE;
508     TYPE tbStockMinimo IS TABLE OF PRODUCTO.stock_minimo%TYPE;
509
510     vtIdProducto    tbIdProducto; -- lista q guarda id de product
511     vtStockAnterior tbStockMinimo;
512     vtStockNuevo    tbStockMinimo;
513     vnIdCategoria   NUMBER := 1;      -- categoria a ajustar
514     vnPorcentaje    NUMBER := 10;     -- aumento del stock minimo, en %
515 BEGIN
516     SELECT id_producto,
517            stock_minimo,
518            CEIL(stock_minimo * (1 + vnPorcentaje / 100))
519     BULK COLLECT INTO vtIdProducto, vtStockAnterior, vtStockNuevo
520     FROM   PRODUCTO
521     WHERE  id_categoria = vnIdCategoria
```

Salida de Script x

Tarea terminada en 0.228 segundos

```
Categoria 1 | aumento de stock minimo: 10% | productos cargados: 4
   Producto 1001: stock minimo 20 -> 22
   Producto 1002: stock minimo 15 -> 17
   Producto 1003: stock minimo 25 -> 28
   Producto 1015: stock minimo 20 -> 22
Filas actualizadas por el FORALL: 4
   En la tabla: producto 1001 ahora hay stock minimo 22
   En la tabla: producto 1002 ahora hay stock minimo 17
   En la tabla: producto 1003 ahora hay stock minimo 28
   En la tabla: producto 1015 ahora hay stock minimo 22

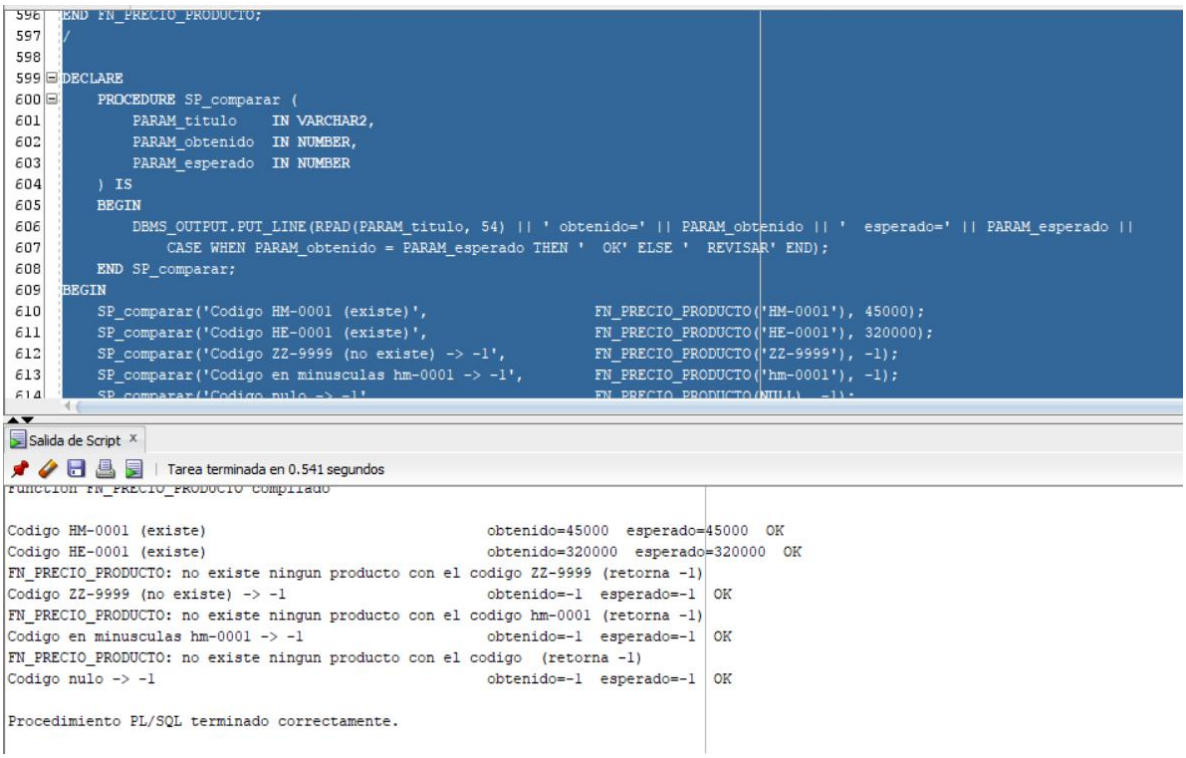
Procedimiento PL/SQL terminado correctamente.
```

Solución

En el 3.5 subimos el stock mínimo de todos los productos de una categoría (en la prueba, +10% en la categoría 1). Primero, con BULK COLLECT, traemos de una sola vez a dos listas en memoria los ids de los productos y su nuevo stock mínimo (el actual subido el porcentaje y redondeado hacia arriba con CEIL). Después, con FORALL, hacemos todos los UPDATE en un solo envío a la base de datos, usando la casilla de cada lista. Imprimimos cuántos productos se cargaron y cuántas filas se actualizaron (4 y 4) y mostramos cómo quedó la tabla (22, 17, 28 y 22). Terminamos con ROLLBACK para poder repetir la prueba.

Parte 4

4.1



```
END FN_PRECIO_PRODUCTO;
/

DECLARE
    PROCEDURE SP_comparar (
        PARAM_titulo    IN VARCHAR2,
        PARAM_obtenido  IN NUMBER,
        PARAM_esperado  IN NUMBER
    ) IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE(RPAD(PARAM_titulo, 54) || ' obtenido=' || PARAM_obtenido || '  esperado=' || PARAM_esperado ||
            CASE WHEN PARAM_obtenido = PARAM_esperado THEN '  OK' ELSE '  REVISAR' END);
    END SP_comparar;
BEGIN
    SP_comparar('Codigo HM-0001 (existe)',                FN_PRECIO_PRODUCTO('HM-0001'), 45000);
    SP_comparar('Codigo HE-0001 (existe)',                FN_PRECIO_PRODUCTO('HE-0001'), 320000);
    SP_comparar('Codigo ZZ-9999 (no existe) -> -1',       FN_PRECIO_PRODUCTO('ZZ-9999'), -1);
    SP_comparar('Codigo en minusculas hm-0001 -> -1',     FN_PRECIO_PRODUCTO('hm-0001'), -1);
    SP_comparar('Codigo nulo -> -1',                      FN_PRECIO_PRODUCTO(NULL), -1);
```

Salida de Script

Tarea terminada en 0.541 segundos

```
Function FN_PRECIO_PRODUCTO compilado

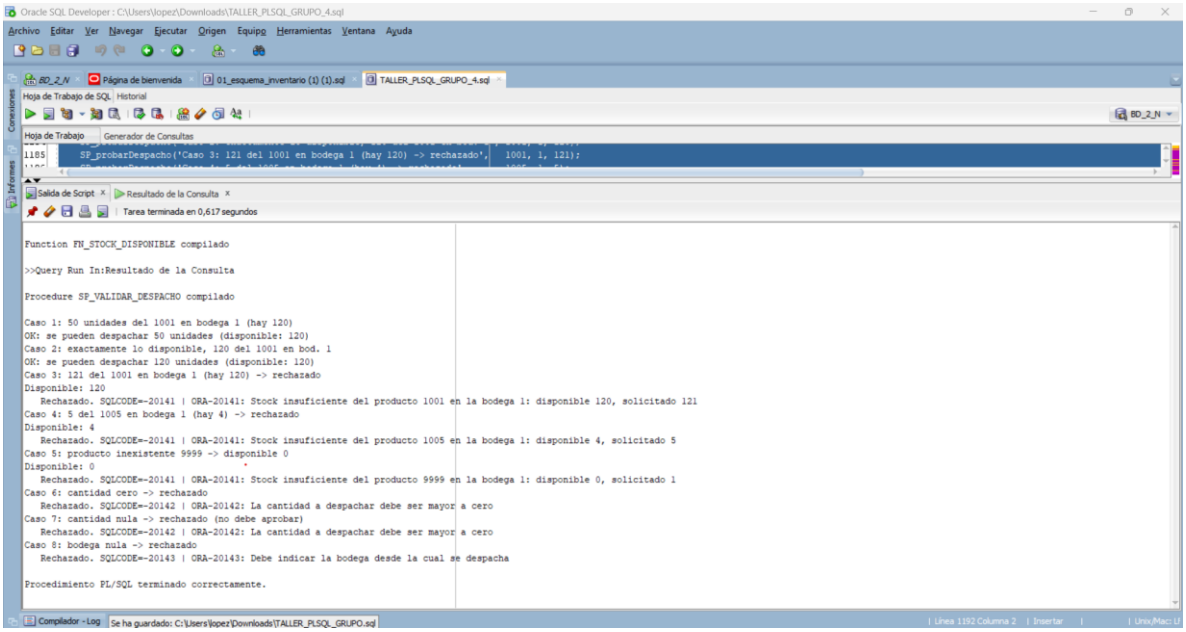
Codigo HM-0001 (existe)                              obtenido=45000  esperado=45000  OK
Codigo HE-0001 (existe)                              obtenido=320000  esperado=320000  OK
FN_PRECIO_PRODUCTO: no existe ningun producto con el codigo ZZ-9999 (retorna -1)
Codigo ZZ-9999 (no existe) -> -1                     obtenido=-1  esperado=-1  OK
FN_PRECIO_PRODUCTO: no existe ningun producto con el codigo hm-0001 (retorna -1)
Codigo en minusculas hm-0001 -> -1                   obtenido=-1  esperado=-1  OK
FN_PRECIO_PRODUCTO: no existe ningun producto con el codigo  (retorna -1)
Codigo nulo -> -1                                    obtenido=-1  esperado=-1  OK

Procedimiento PL/SQL terminado correctamente.
```

Solución

En el 4.1 creamos una función que recibe el código de un producto y devuelve su precio. Dentro hace un SELECT ... INTO buscando el producto por código. Si lo encuentra, devuelve el precio. Si no existe, se activa NO_DATA_FOUND: la función imprime un mensaje y devuelve -1. Si hubiera más de un producto con ese código, se activaría TOO_MANY_ROWS: imprime un mensaje y devuelve -2. En la prueba, un código existente devuelve su precio (HM-0001 da 45.000), y un código que no existe, uno escrito en minúsculas o uno nulo devuelven -1. El caso -2 no se puede provocar porque el código tiene restricción UNIQUE.

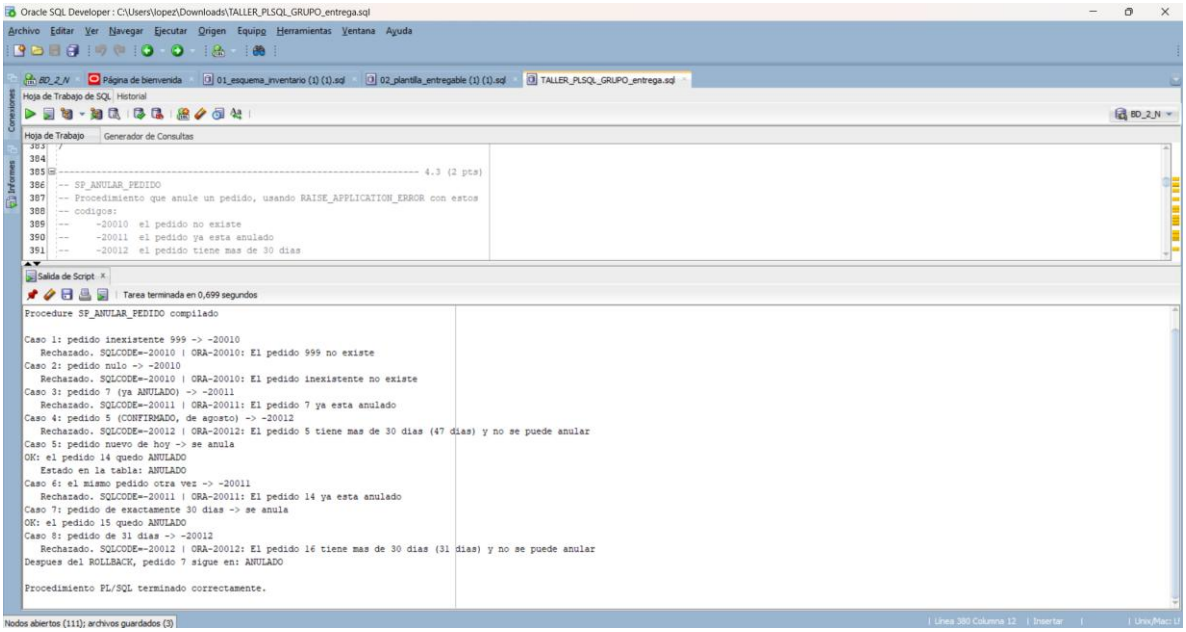
4.2



Solución

En el 4.2 creamos un procedimiento que dice si se puede despachar una cantidad de un producto desde una bodega. Primero revisa que se indique la bodega y que la cantidad sea mayor que cero. Luego consulta el disponible con FN_STOCK_DISPONIBLE. Si la cantidad pedida supera el disponible, levanta con RAISE una excepción propia que declaramos, e_stock_insuficiente. En el bloque EXCEPTION la capturamos, mostramos cuánto hay disponible y devolvemos el error -20141 con RAISE_APPLICATION_ERROR. Si alcanza el stock, imprime que se puede despachar. En la prueba se aprueban 50 y 120 unidades de un producto con 120, y se rechazan 121 unidades, un producto sin stock suficiente, un producto inexistente, la cantidad en cero, la cantidad nula y la bodega nula.

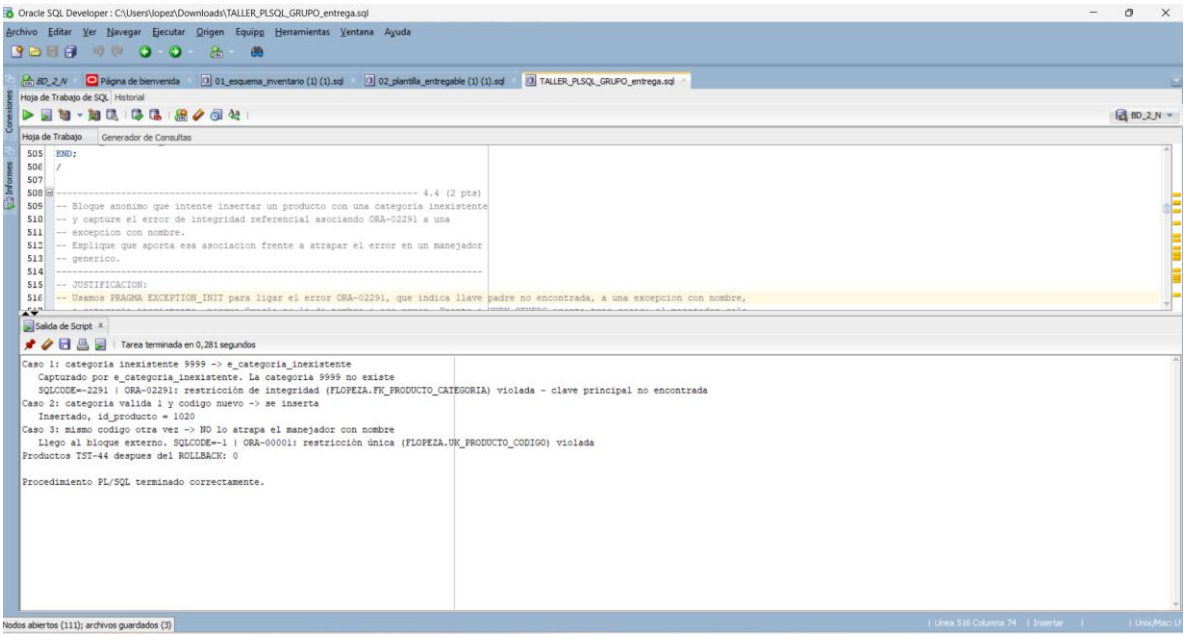
4.3



Solución

En el 4.3 creamos el procedimiento SP_ANULAR_PEDIDO con los tres codigos propios que pide el enunciado. Primero lee y bloquea el pedido con SELECT FOR UPDATE y, si no existe, responde -20010. Luego revisa si ya esta anulado y responde -20011, y al final compara la antiguedad en dias de calendario y responde -20012 si pasa de 30 dias. Solo si las tres validaciones se cumplen cambia el estado a ANULADO. En la captura se ven los 8 casos: pedido inexistente, nulo, ya anulado y de agosto rechazados, un pedido de hoy y uno de exactamente 30 dias anulados, y el de 31 dias rechazado. Al final el ROLLBACK deja el pedido 7 como estaba. En el comentario tambien explicamos la diferencia entre RAISE y RAISE_APPLICATION_ERROR.

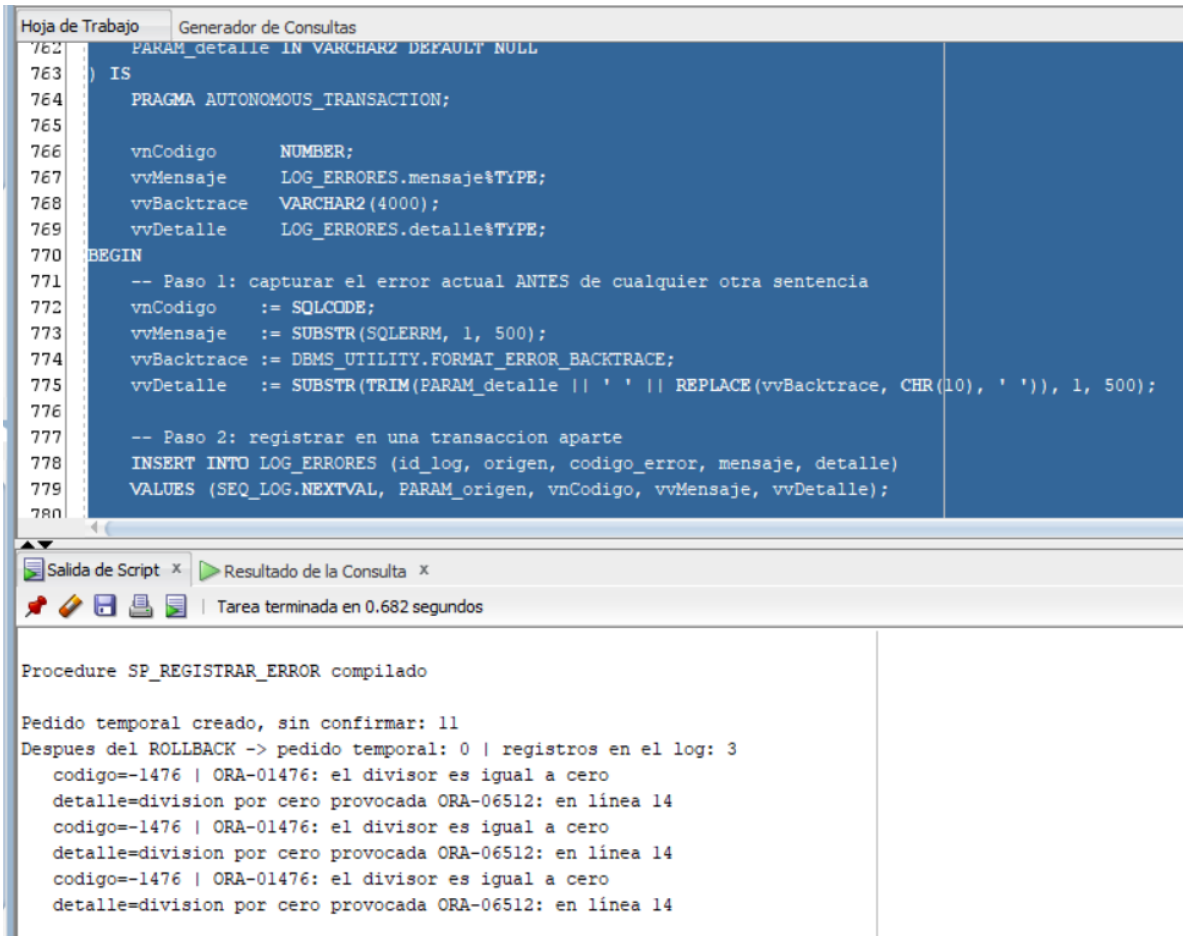
4.4



Solución

En el 4.4 declaramos la excepcion e_categoria_inexistente y la asociamos al error ORA-02291 con PRAGMA EXCEPTION_INIT. Un procedimiento local intenta insertar un producto y captura solo esa excepcion. En el caso 1 usamos la categoria 9999, que no existe, y el bloque la atrapa por su nombre y muestra el SQLCODE -2291. En el caso 2 la categoria es valida y el producto se inserta. En el caso 3 repetimos el codigo del producto y el error ORA-00001 no es atrapado por el manejador con nombre, sino que llega al bloque externo sin disfrazarse. Esa es la ventaja frente a WHEN OTHERS. Al final el ROLLBACK borra los productos de prueba.

4.5

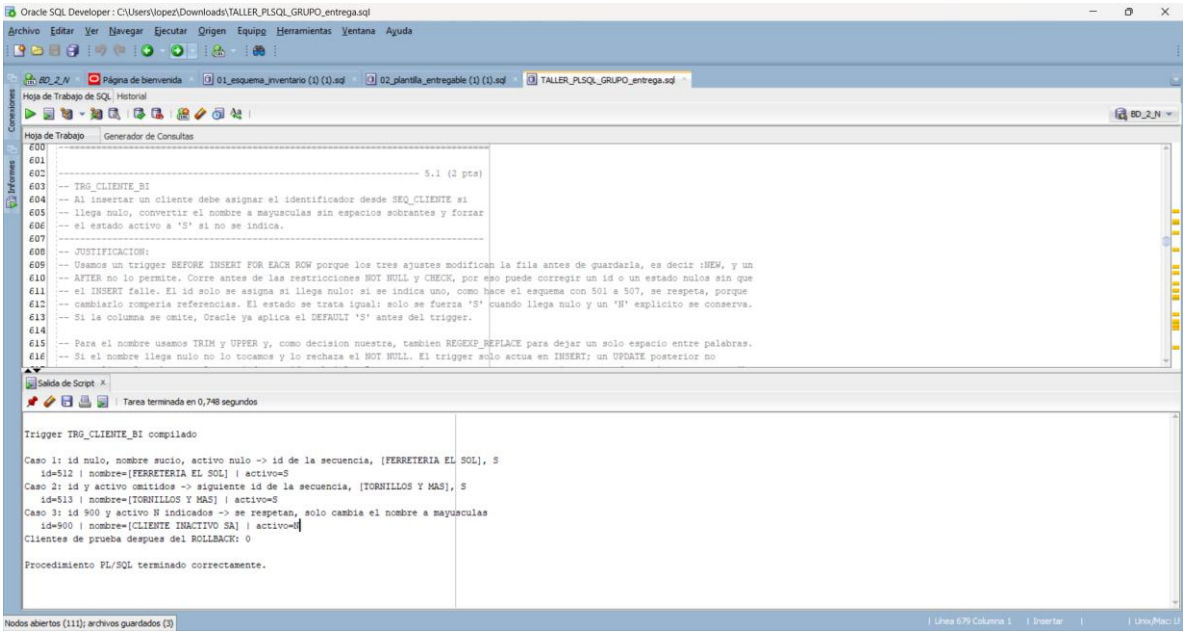


Solución

En el 4.5 creamos un procedimiento que guarda un error en la tabla LOG_ERRORES. Recibe el origen y, opcionalmente, un detalle. Al inicio copia a variables el SQLCODE, el SQLERRM y el rastro del error, y los inserta en el log junto con el origen. Está marcado con PRAGMA AUTONOMOUS_TRANSACTION, así que corre en una transacción aparte y su COMMIT solo confirma el registro del log. En la prueba creamos un pedido temporal, provocamos una división por cero, llamamos al procedimiento desde el manejador y hacemos ROLLBACK. El resultado muestra que el pedido temporal desapareció (0), pero el registro del error quedó guardado (1), con su código y su mensaje.

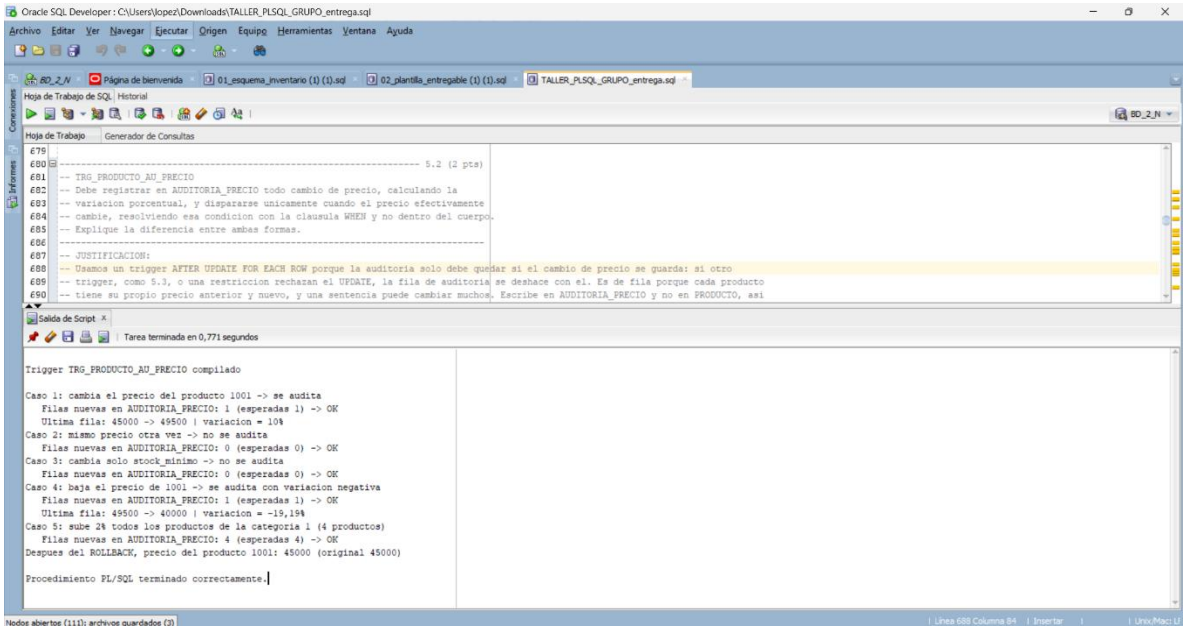
Parte 5

5.1



Solución

En el 5.1 creamos el trigger TRG_CLIENTE_BI como BEFORE INSERT por fila, porque tiene que modificar los valores antes de guardar la fila. Asigna el id desde SEQ_CLIENTE solo si llega nulo, deja el nombre en mayusculas sin espacios al inicio, al final ni repetidos en medio, y pone el estado en S cuando llega nulo. En la captura, el caso 1 inserta un cliente con id nulo, nombre sucio y estado nulo, y queda con un id de la secuencia, el nombre limpio y S. El caso 2 omite el id y el estado y da el mismo resultado. En el caso 3 indicamos id 900 y estado N, y el trigger los respeta y solo cambia el nombre a mayusculas. El ROLLBACK deja el conteo de clientes de prueba en 0.

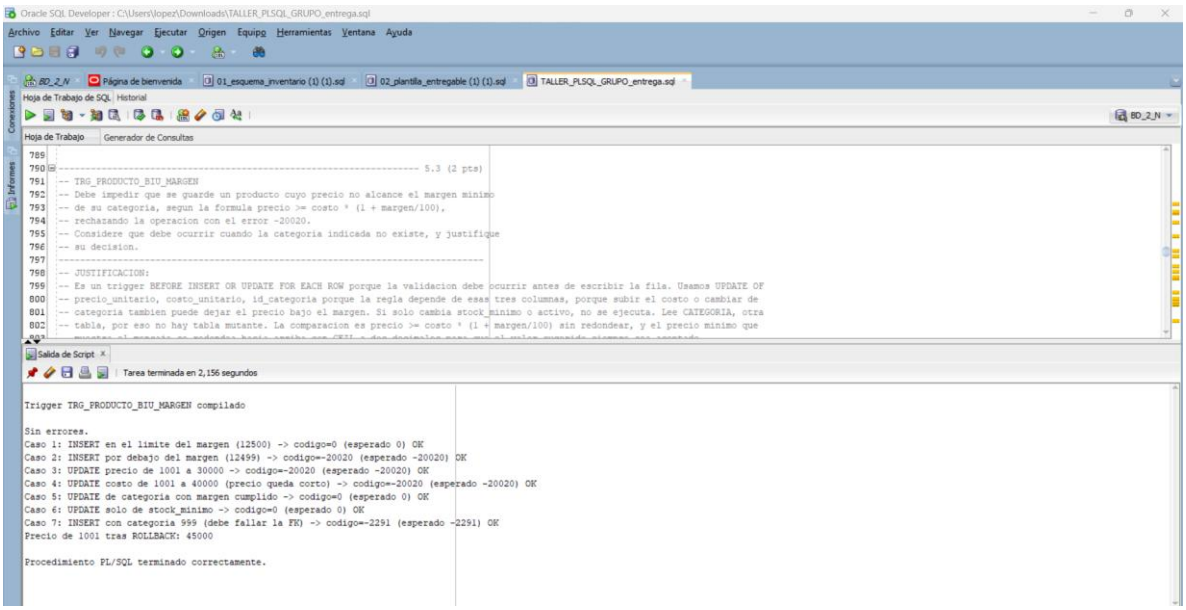


5.2

Solución

En el 5.2 creamos el trigger TRG_PRODUCTO_AU_PRECIO como AFTER UPDATE por fila sobre la columna del precio, con la clausula WHEN para que solo se dispare cuando el precio realmente cambia. Registra en AUDITORIA_PRECIO el precio anterior, el nuevo y la variacion porcentual. En la captura, el caso 1 sube el precio del producto 1001 y se audita con variacion de 10. El caso 2 escribe el mismo precio y el caso 3 cambia solo el stock minimo, y en los dos no se audita nada. El caso 4 baja el precio y la variacion sale negativa. El caso 5 sube 2 por ciento a toda la categoria 1 y queda una fila de auditoria por producto. Todos salen con OK y el ROLLBACK devuelve el precio original.

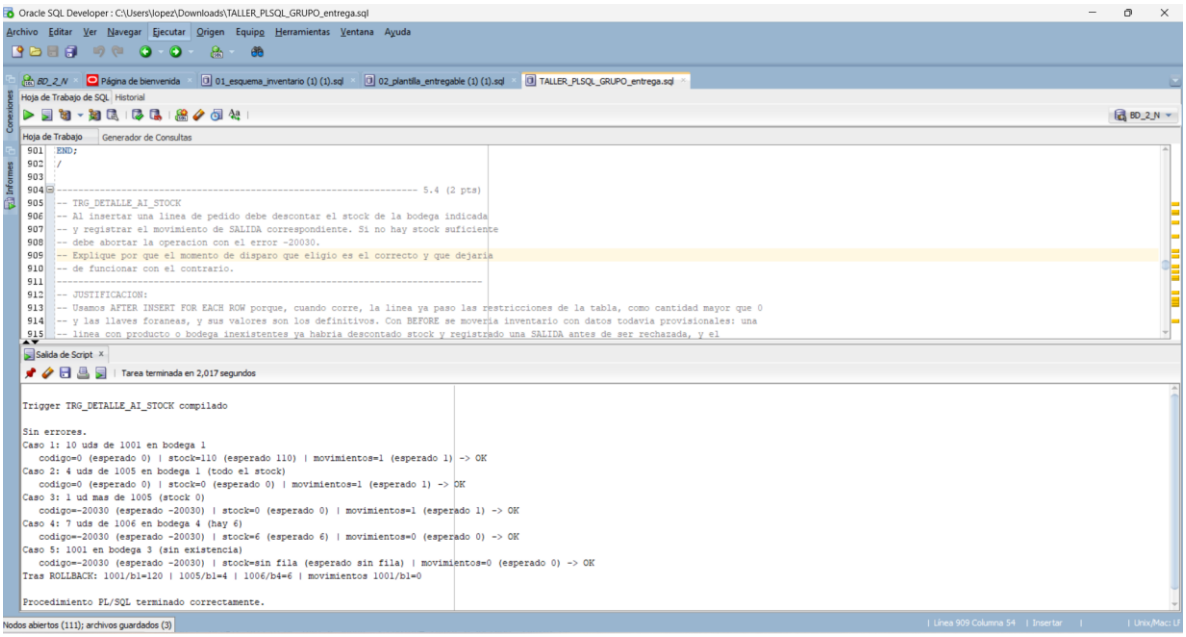
5.3



Solución

En el 5.3 creamos el trigger TRG_PRODUCTO_BIU_MARGEN, BEFORE INSERT OR UPDATE por fila, que rechaza con el error -20020 los productos cuyo precio no alcanza el costo por uno mas el margen minimo de su categoria. Se dispara cuando cambia el precio, el costo o la categoria. En la captura, el caso 1 inserta un producto justo en el limite del margen y se acepta, y el caso 2, un peso por debajo, se rechaza. Los casos 3 y 4 muestran el rechazo al bajar el precio y al subir el costo. Los casos 5 y 6 pasan, porque el margen se cumple o no se toca el precio. En el caso 7 la categoria no existe y el trigger no lanza -20020, sino que deja que la llave foranea responda ORA-02291. Al final el ROLLBACK devuelve el precio original.

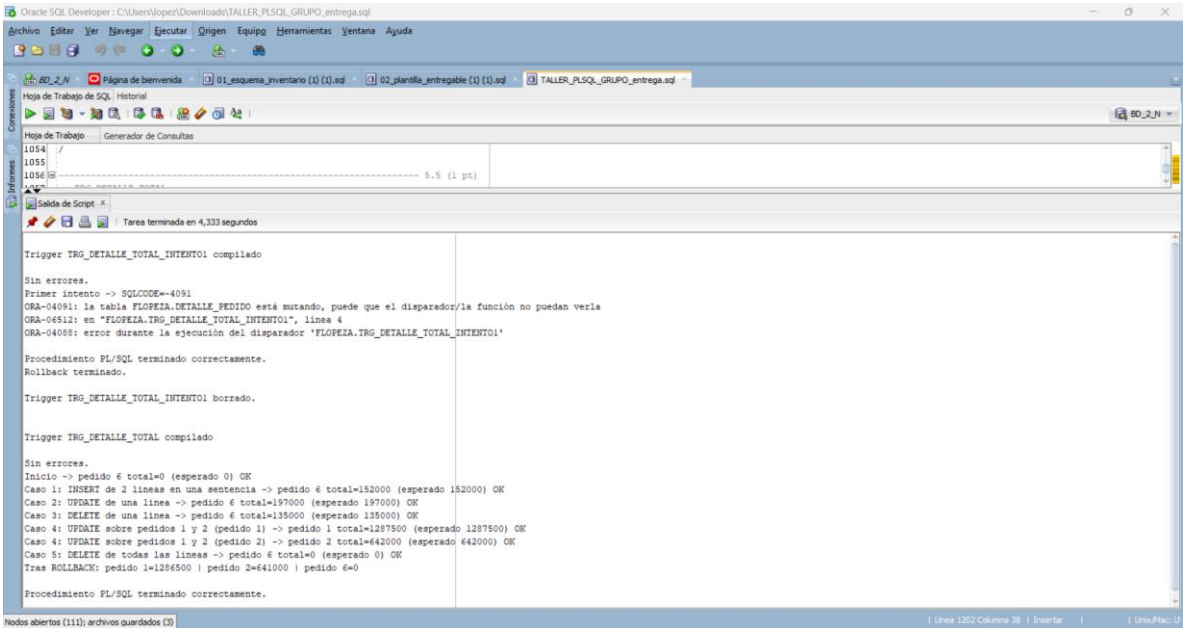
5.4



Solución

En el 5.4 creamos el trigger TRG_DETALLE_AI_STOCK, AFTER INSERT por fila sobre DETALLE_PEDIDO. Descuenta el stock de la bodega con un solo UPDATE que exige cantidad suficiente y registra la SALIDA en MOVIMIENTO_INVENTARIO. Si el UPDATE no toca ninguna fila, lanza el error -20030. En la captura, el caso 1 vende 10 unidades del producto 1001 y el stock baja de 120 a 110 con un movimiento. El caso 2 vende las 4 unidades que hay del producto 1005 y queda en 0. Los casos 3, 4 y 5 se rechazan con -20030: vender una unidad mas con stock 0, pedir 7 cuando hay 6 y vender un producto en una bodega donde no existe. Al final el ROLLBACK deja el stock como estaba.

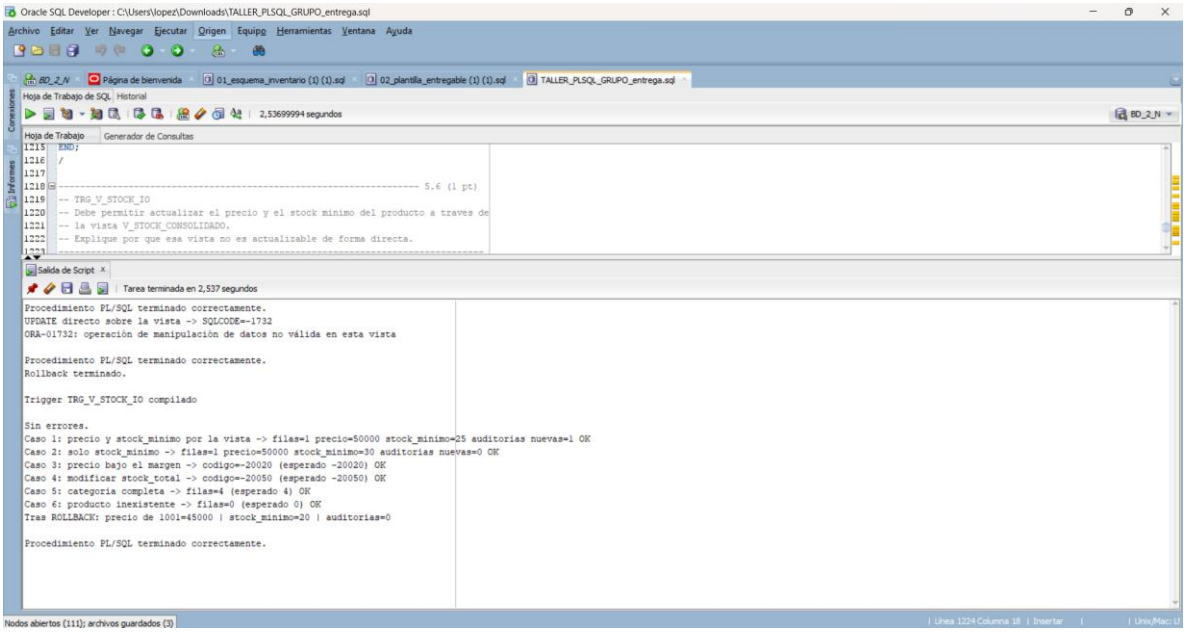
5.5



Solución

En el 5.5 primero probamos un trigger normal por fila que recalcula el total con un SELECT sobre DETALLE_PEDIDO. Compila bien, pero al ejecutar un UPDATE falla con ORA-04091, tabla mutante, porque la tabla se esta modificando y el trigger no puede leerla. En la captura se ve primero ese error como evidencia. Lo resolvimos con TRG_DETALLE_TOTAL, un COMPOUND TRIGGER. En AFTER EACH ROW solo anota en una coleccion el pedido de cada linea tocada y en AFTER STATEMENT, cuando la tabla ya no esta mutando, recalcula el total de cada pedido una sola vez. Despues, en la misma captura, el pedido 6 empieza en 0 y sus totales cambian bien con un INSERT de dos lineas, un UPDATE, un DELETE, una sentencia que toca dos pedidos y un DELETE de todas las lineas que deja el total en 0. Todos los casos salen con OK.

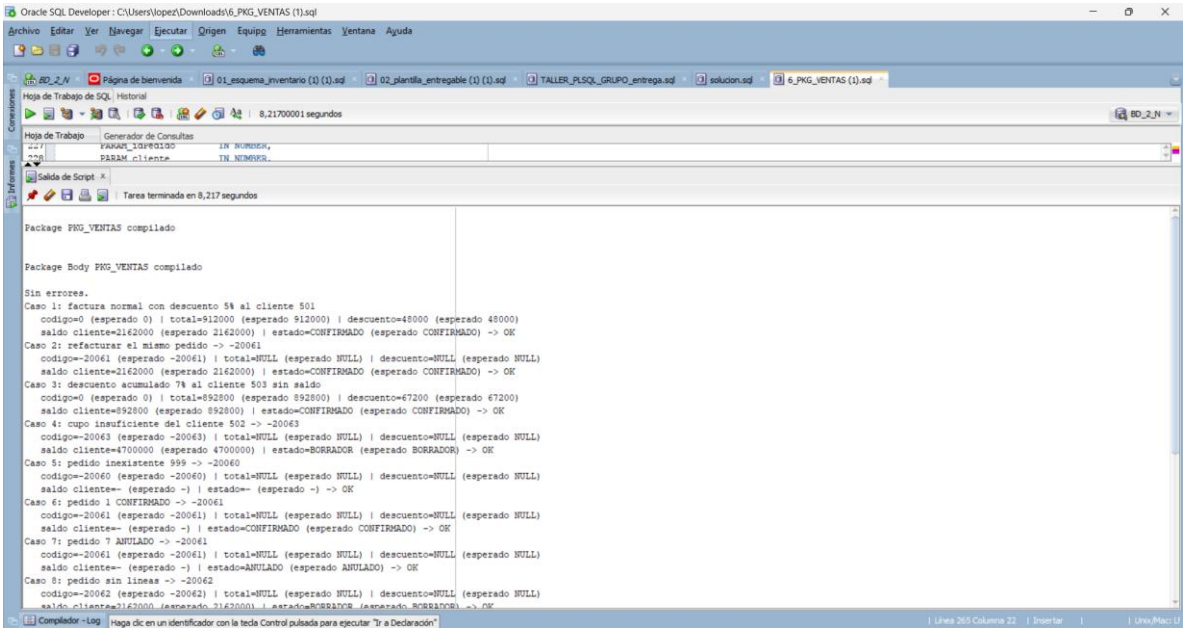
5.6

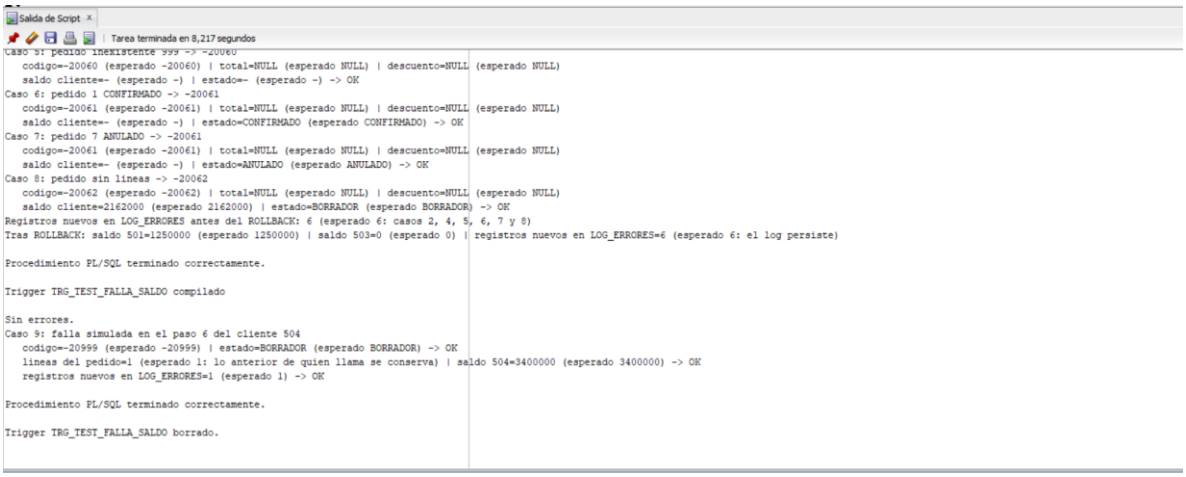


Solución

En el 5.6 la vista V_STOCK_CONSOLIDADO une varias tablas y agrupa para calcular el stock total, por eso un UPDATE directo falla con ORA-01732. En la captura se ve primero ese error. Para permitir el cambio creamos TRG_V_STOCK_IO, un INSTEAD OF UPDATE por fila que actualiza PRODUCTO usando el id del producto. Solo deja cambiar el precio y el stock minimo, y si se toca otra columna responde -20050. Despues, en la misma captura, el caso 1 cambia precio y stock minimo y se audita el cambio de precio. El caso 2 cambia solo el stock minimo y no audita. En el caso 3, un precio bajo el margen es rechazado por el trigger 5.3 con -20020, y en el caso 4 intentar cambiar el stock total se rechaza con -20050. El caso 5 actualiza una categoria completa y toca 4 filas, y en el caso 6 un producto inexistente toca 0 filas sin error.

Parte 6 (Reto integrador)





Solución

En el 6 creamos el paquete PKG_VENTAS con el procedimiento SP_FACTURAR_PEDIDO, que une los objetos de las partes anteriores. Primero bloquea el pedido y comprueba que exista y este en BORRADOR, luego bloquea al cliente y suma las lineas con un cursor. El descuento lo calcula FN_CALCULAR_DESCUENTO y el cupo lo valida FN_CUPO_DISPONIBLE, siempre antes de subir el saldo, porque si no el cliente sin saldo perderia el 2% adicional. Si todo sale bien, deja el total neto, pasa el pedido a CONFIRMADO y suma ese total al saldo del cliente. Todo parte de un SAVEPOINT: ante cualquier error se registra con SP_REGISTRAR_ERROR, se revierte solo lo que hizo el procedimiento y se relanza el error. El COMMIT lo hace quien llama, no el paquete. Las dos capturas son seguidas y muestran todo el log de esta parte. En la primera se ve la compilacion del paquete, el caso 1 con descuento del 5% y el caso 3 con el 7% acumulado, y los casos 2 y 4 a 8 con los codigos -20061, -20063, -20060 y -20062. La segunda continua desde el caso

5 y termina con el caso 9, que simula una falla al subir el saldo del cliente 504: el pedido vuelve a BORRADOR sin perder sus lineas y el log guarda los errores aunque las pruebas terminaron en ROLLBACK.